

Deeper Dive Into Express Applications

CS5610 Summer 2026 Midterm



What is Express?

- ❑ Express is a web app framework built off of Node JS
- ❑ It helps servers:
 - ❑ Receive requests
 - ❑ Decide what to do with them
 - ❑ Communicate with databases
 - ❑ Send responses

```
const express = require('express'); // import
const app = express(); // create app
app.listen(3000); // start the web server
```

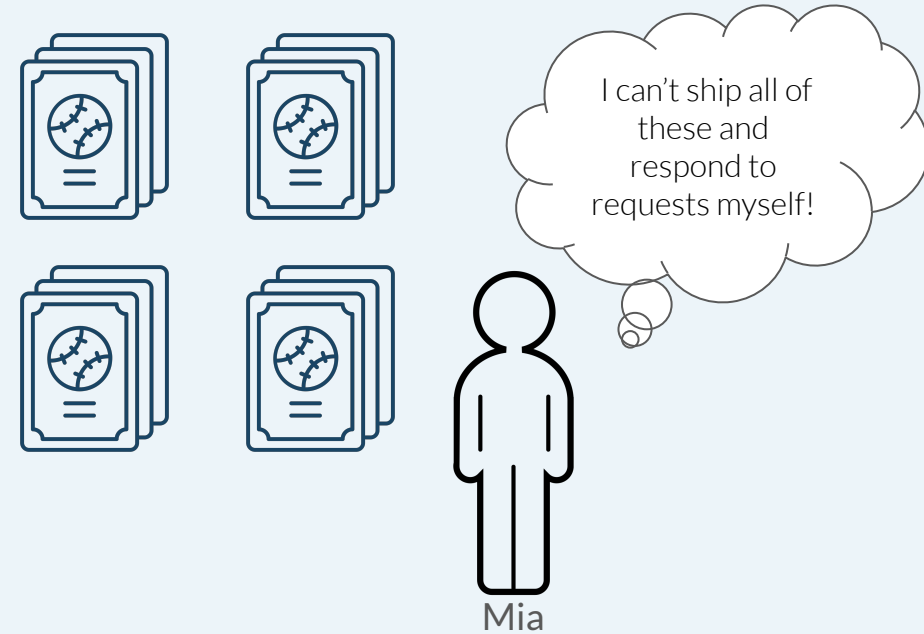
... Okay, but what does this actually look like?

Let's start with a story about Mia and Owen...



Mia owns a small store for selling baseball cards, and Owen wants to buy one

Mia gets tons of direct requests online to buy cards, and she is getting overwhelmed

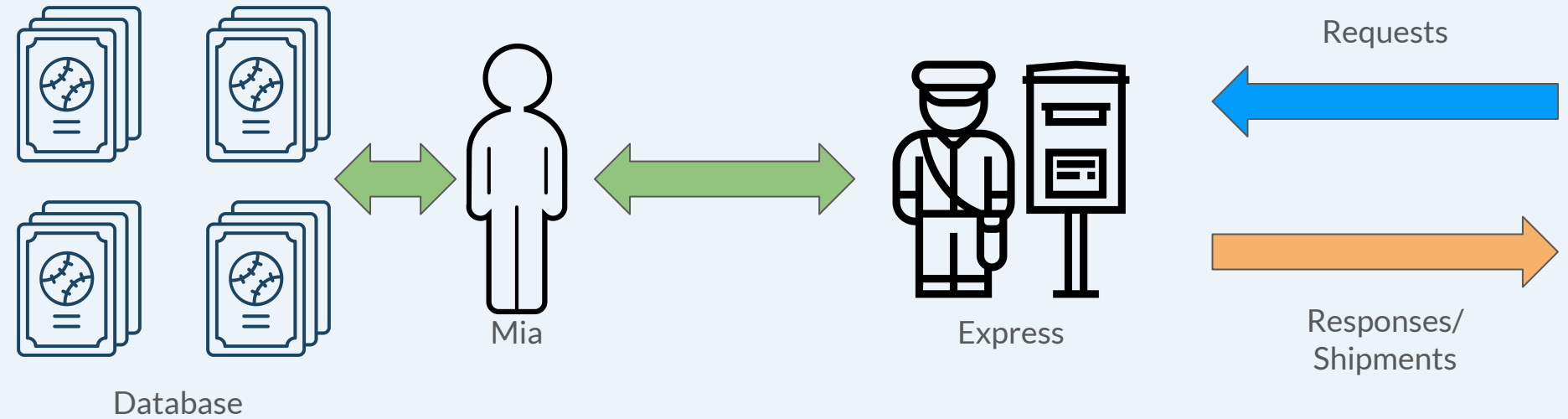


Owen wants to know if she has an Oakland A's card

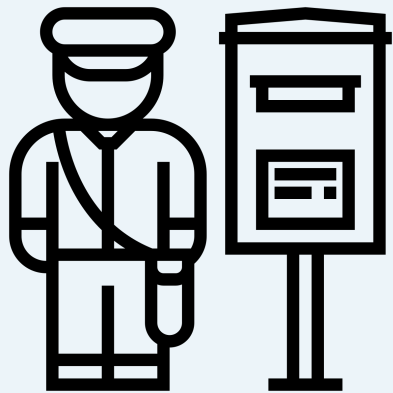


They are 3,000 miles apart! Owen doesn't know if she has the card, or how much it would cost, and he can't go to the physical store directly.

Mia hires an assistant Express to handle all the requests and shipments...

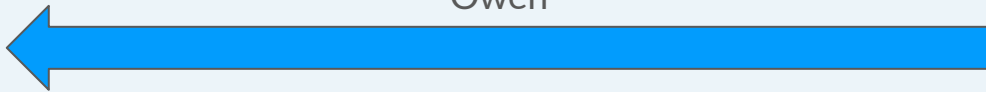


Express sits between the client (Owen) and the server (Mia), so now Mia only needs to talk to one person



Express

"I will give you \$20 for an Oakland A's card of any player" –
Owen

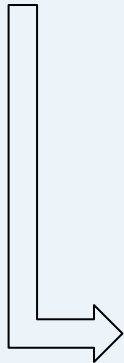


"Sorry, that isn't right. Our Oakland A's card costs \$25" –
Express

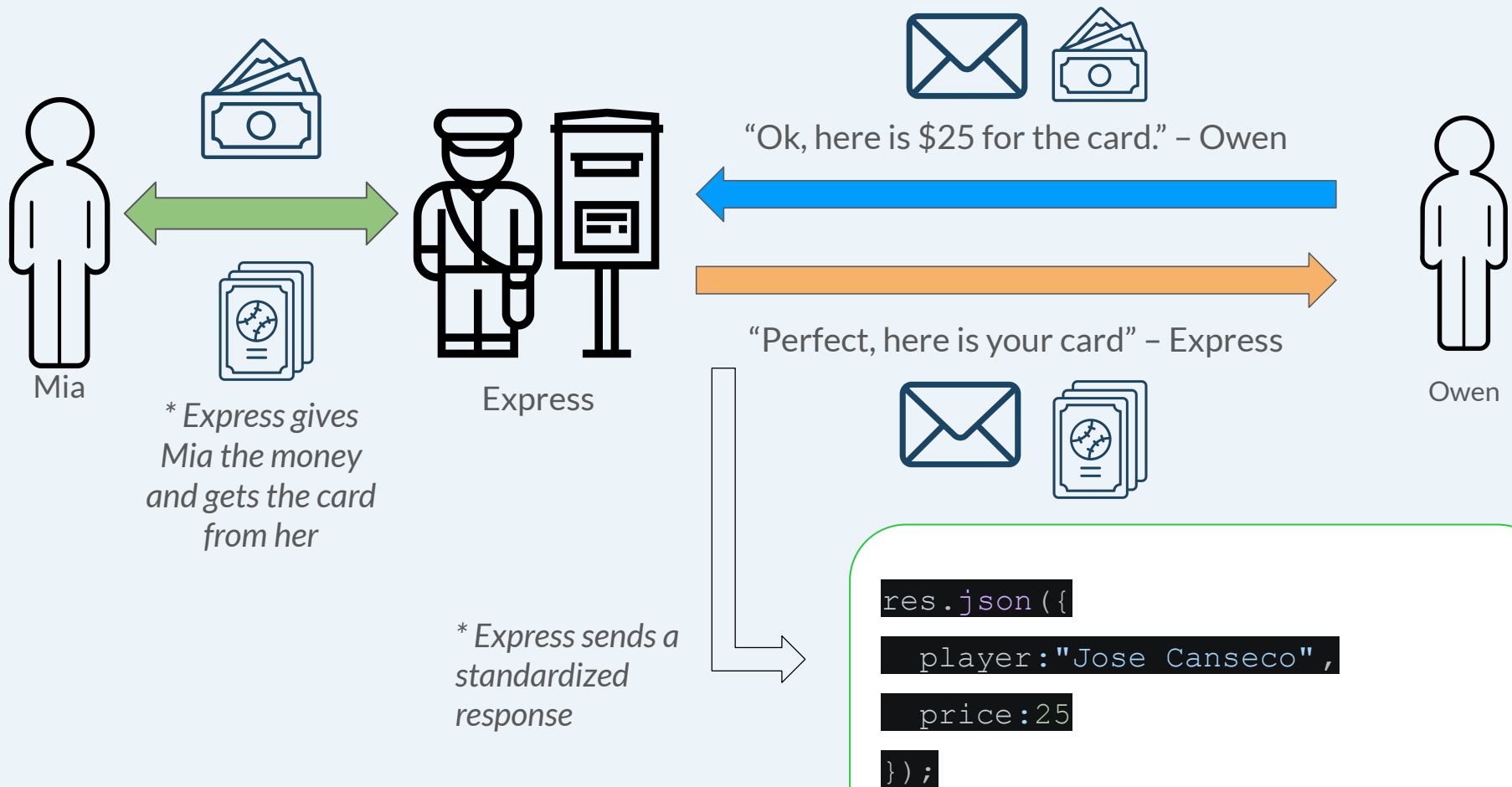


Owen

*Express checks if
requests are valid
before sending them to
Mia*

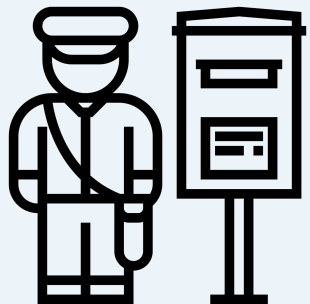


```
app.post('/purchase', (req, res) => {  
  if(req.body.price < 25){  
    return res  
      .status(400)  
      .send("Invalid amount");  
  }  
});
```



Now the transaction is complete!

How did Express know what to do?



Express has routes

```
app.get('/cards/oakland', getCard)
```

```
app.post('/purchase', buyCard)
```

This is kind of like having a lookup table:

Request	Destination
GET /cards/oakland	getCard()
GET /purchase	buyCard()

Express Routing

Express looks at:

HTTP Method

- GET
- POST
- DELETE
- PUT

URL

- /cards/oakland
- /cards/redsox
- Etc

to choose the appropriate
callback function

```
app.get('/cards/:team', (req, res) => {  
  const team = req.params.team;  
  res.send(team);  
});
```

Express sees GET /cards/{team} and executes
(req,res)=>{}

Routing

Instead of

```
app.get('/cards')  
app.get('/users')  
app.get('/orders')
```

In one file, we separate the routes ->

This helps us organize larger applications

```
// cards.js
```

```
const router = express.Router();
```

```
router.get('/');
```

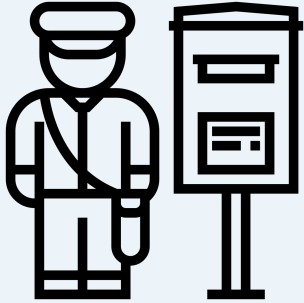
```
router.post('/');
```

```
module.exports = router;
```

```
// backend.js
```

```
app.use( '/cards', cardsRouter);
```

How did Express send the baseball card?

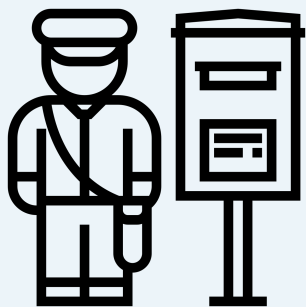


Express

IRL, Express sends the data from the database. It can package a response using `res.json()`, `res.send()`, etc.

Express also parses incoming data using `app.use(express.json());` and `req.body`

What else can Express do as Mia's assistant?



Express

Organize

Express can use routers [ex: `app.use('/cards', cardsRouter);`] to keep apps organized

Security

Express can deny invalid requests (400), unauthorized users (401), and forbid content from being accessed (403)

Express is not just a messenger, it can also be a security guard, a receptionist, package inspector, translator, and delivery driver.

Anatomy of An Express Application

Components

app	Express application
routes	Match URLs
req	Incoming request
res	Outgoing response
Callback functions	Handlers, execute logic

```
const express = require('express');
```

```
const app = express();
```

```
app.use(express.json());
```

```
app.get('/', (req, res) => {
```

```
  res.send("Hello");
```

```
});
```

```
app.listen(3000);
```

HTTP Methods

Express supports GET, POST, PUT, DELETE, and more

GET	Retrieve information	<code>app.get('/cards/')</code>
PUT	Replace existing data	<code>app.put('/cards/:id')</code>
POST	Create data	<code>app.post('/cards')</code>
DELETE	Remove data	<code>app.delete('/cards/:id')</code>

Route Parameters

Parameters let routes become dynamic

1. Example request: /cards/redsox
2. Express gets req.params.team
 - a. Value is redsox
3. You can use multiple parameters

```
app.get('/cards/:team',
```

```
(req, res) => {
```

```
  const team = req.params.team;
```

```
  res.send(team);
```

```
});
```

Error Handling

Express lets us catch errors and send meaningful responses back to the client in the route:

```
app.get('/cards/:id', async (req, res) => {  
  const card = await db  
    .collection('cards')  
    .findOne({  
      id: req.params.id  
    });  
  
  if (!card) {  
    return res  
      .status(404)  
      .json({  
        message: "Card not found"  
      });  
  }  
  
  res.json(card);  
});
```


Connecting Express to MongoDB

Import mongo

```
import { MongoClient, ObjectId, ServerApiVersion } from "mongodb";
```

Import for env variables

```
import "dotenv/config";
```

Database URI

```
const uri = process.env.MONGO_URI;
```

Set client

```
const client = new MongoClient(uri);
```

Connect to client

```
await client.connect();
```

Log connection for error checking

```
console.log(  
  "Connected to MongoDB"  
);
```

Access a database

```
const db = client.db("cardsDB");
```

Access a collection

```
const cards = db.collection("cards");
```

Resources

- Iconography: [Noun Project](#) (subscription)
- Express Documentation:
 - [Introduction](#)
 - [Routing](#)
- [Connect to MongoDB](#)